

Lab 2 - Cloud Data, Stat 214, Spring 2026

March 21, 2026

1 Introduction

Cloud detection plays a critical role in climate science, since clouds retain heat and reflect solar radiation simultaneously to regulate global temperature. Arctic cloud detection is challenging because of the visual similarity between clouds and sea ice, both appearing white and cold. The Multi-angle Imaging SpectroRadiometer (MISR) overcomes this challenge by capturing the same scene from nine different angles. As clouds are situated at higher altitudes than the surface of the earth, they appear to shift positions across different camera angles, while the surface remains fixed.

In this report, we build a prediction model to distinguish cloud using MISR data. The primary challenge is scarcity of label: only 3 of the 164 provided images have expert labels. To address this, our analysis proceeds as follows. First, in the Data section, we give an overview of the dataset and its features. Second, in the EDA section, we analyze the angle radiance relationships, split the data to check generalizability, and examine data quality. Third, in the Feature Engineering section, we select the best features, construct new features, and perform transfer learning using an autoencoder. Finally, in the Predictive Modeling section, we train and compare multiple classifiers. Then, we select the best model, conduct post-hoc error analysis on it, and assess its stability and generalizability.

2 Data

The dataset contains 164 satellite images. While most of these images are unlabeled, three images (orbits O013257, O013490, and O012791) contain expert labels which we can use for training and evaluation. Each row in an image corresponds to a pixel in the satellite image, which itself represents a 275 meter by 275 meter region on the ground. The columns can be described as follows:

- **Coordinates:** Each row has a unique (x,y) pair that determines where the pixel is located in the image. Each image is roughly 304 pixels wide and 382 pixels high.
- **NDAI:** Normalized Difference Angular Index; measures forward scattering of light by comparing radiance measurements from the Df angle with the An angle.
- **SD:** Standard deviation of radiance at the An angle within an 8×8 group of pixels.
- **CORR:** Correlation; measures the linear correlation between different MISR viewing angles.
- **Radiance:** Red band radiance measurements from five sensor angles: Df, Cf, Bf, Af, and An.
- **Expert label:** +1 (cloud), -1 (not cloud), or 0 (unlabeled).

The data is highly relevant to our task of identifying clouds in the Arctic because it comes from the same type of satellite observations we will use in the future. Although the data is over 20 years old, it is still valid since the fundamental principles of radiation scattering by clouds and ice has not changed.

One thing to take into consideration is that the expert labels are based off human judgment, where definite labels are only given for pixels that the expert is very confident in. Therefore, a large portion of the 'labeled' data is actually unlabeled.

3 EDA

3.1 Visualizing Expert Labels

We begin by visualizing the labeled data to get a sense of how the clouds appear. Since there are only three labeled images in our dataset, it makes sense to visualize all three of them and look for cloud patterns and

similarities. Although labels are based on human judgment and may not tell the whole truth, they still provide a good starting point to understand the data and the classification problem.

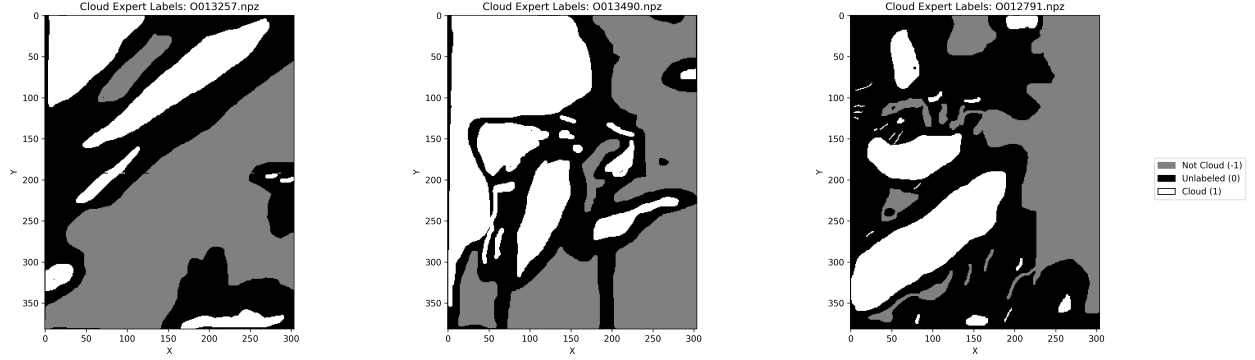


Figure 1: Visualization of expert labeled images

Note: While no explicit missing values exist, a few pixel coordinates are completely absent from the dataset. This was most apparent on the left border of the images. When making Figure 1, we filled these to form rectangular images and marked them as unlabeled (0).

3.2 Correlation of Features

To identify predictive features and avoid multicollinearity issues during modeling, we analyzed correlations between variables using heatmaps (Figure 2).

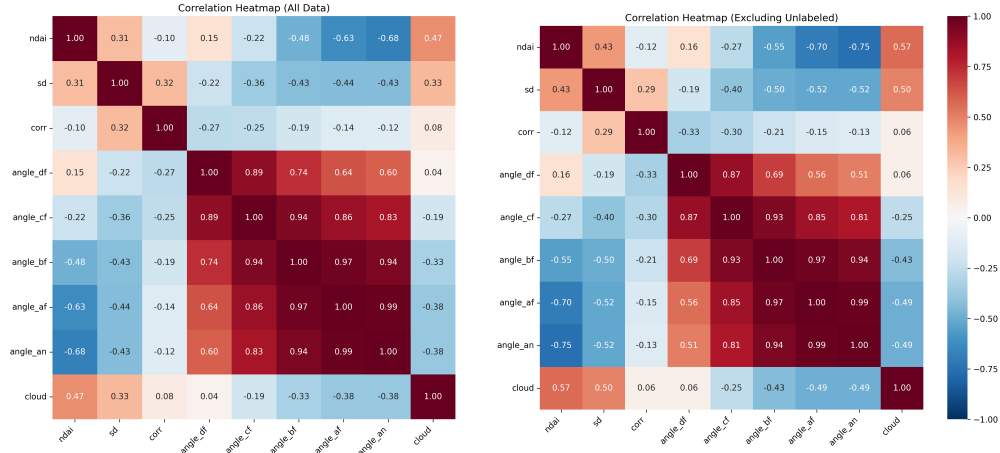


Figure 2: Correlation heatmaps for the full dataset (left) and labeled subset (right)

As expected, the radiance measurements at different angles are strongly correlated with each other. Engineered features such as NDAI and SD also correlate moderately with radiance, which makes sense because these were derived from measurements taken at different angles. In both panels, NDAI has a moderate positive correlation with cloud label and appears to be the highest correlated feature. When focusing on labeled pixels only, SD also becomes moderately positively correlated with cloud label, and the radiance angles all become more strongly correlated with cloud label, especially for more overhead angles.

3.3 Exploring Radiance

We further examined the relationship between radiance measurements and cloud labels across sensor angles, ranging from the Df angle (70.5°) to the directly overhead An angle (0.0°). Figures 3 through 5 compare

radiance distributions.

Note: Missing pixels were left as NA since we could not give them a radiance measurement accurately.

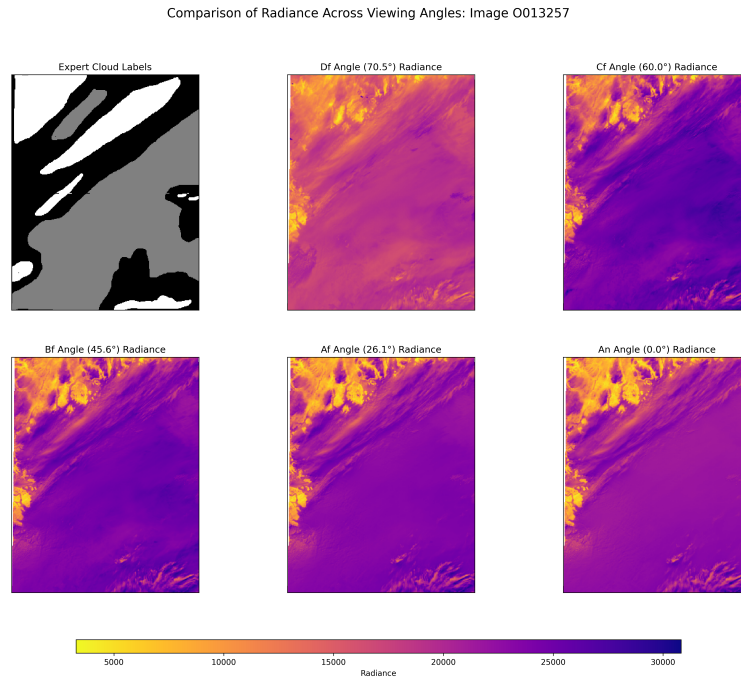


Figure 3: Radiance plot for Image O013257

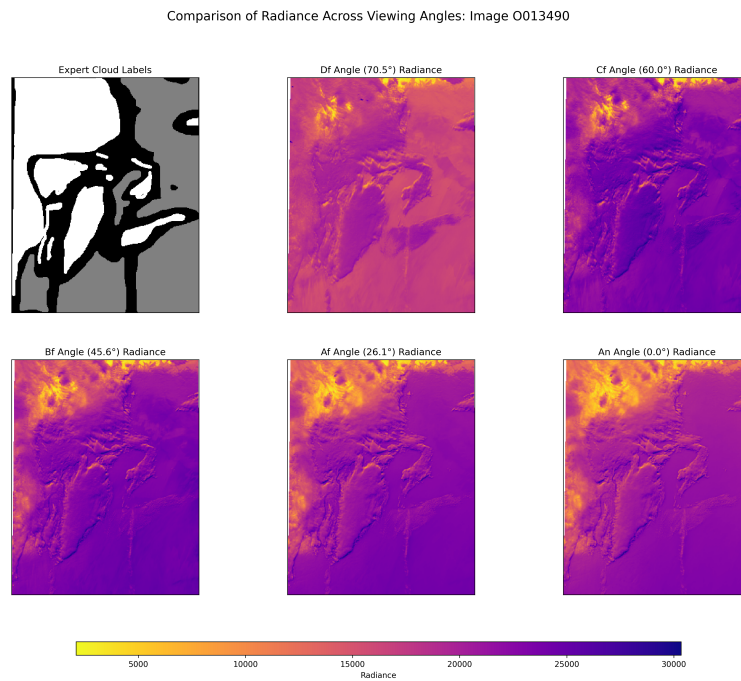


Figure 4: Radiance plot for Image O013490

Comparison of Radiance Across Viewing Angles: Image O012791

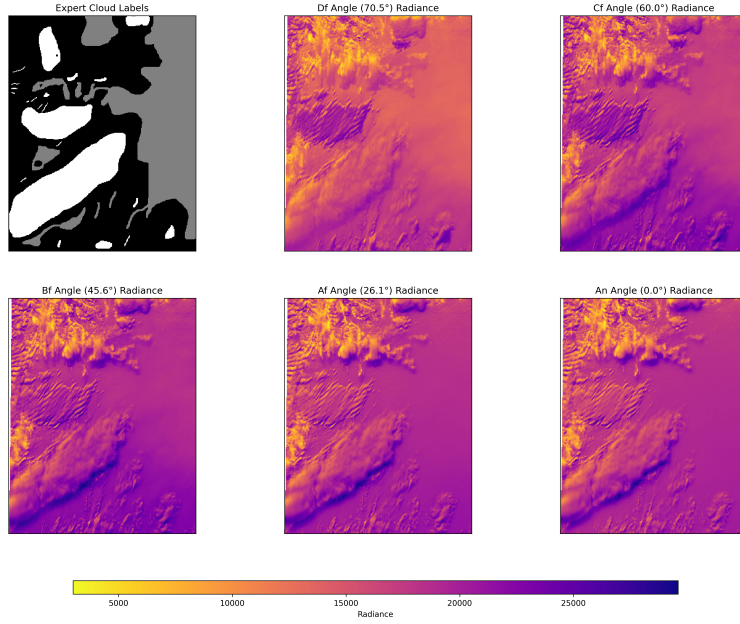


Figure 5: Radiance plot for Image O012791

The boundaries between cloudy and clear areas appear as ridges on the plot. The distinction between areas is obvious. **Cloudy areas tend to have a rougher texture and higher radiance variability, while clear areas have a much smoother appearance.** This may arise because clouds tend to scatter light to a greater degree. Additionally, **the distinction becomes more noticeable as the sensor angle decreases.** The boundaries are sharpest and most distinguishable at the An angle compared to the Df angle.

We further quantify these observations by comparing radiance distributions across angles.

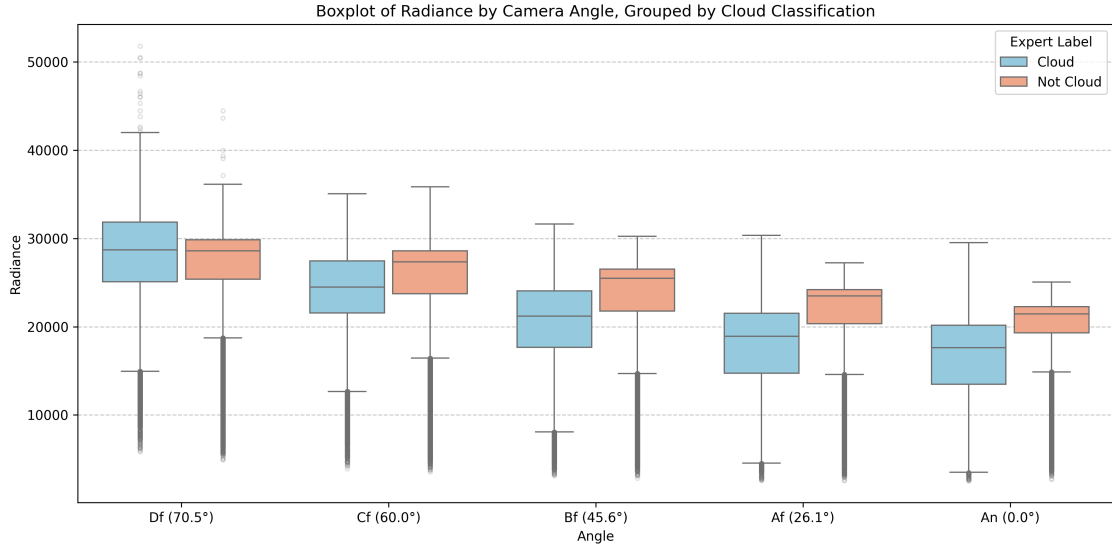


Figure 6: Distribution of radiance for each MISR angle

The grouped boxplots in Figure 6 confirm the visual observations we made previously. Starting from the

As the sensor angle decreases, the radiance measurements also decrease for both cloud and non-cloud pixels. However, cloud pixels experience a steeper decline in radiance. As a result, lower angles see a lower median radiance measurement for cloud pixels relative to non-cloud pixels. In addition, the spread in radiance measurements for cloud pixels does not change much across angles. However, radiance measurements for non-cloud pixels becomes much less spread out at lower angles.

3.4 Exploring Engineered Features

The dataset contains three engineered features. As mentioned before, NDAI has the strongest correlation with cloud label, so we take a closer look at how NDAI is distributed between classes.

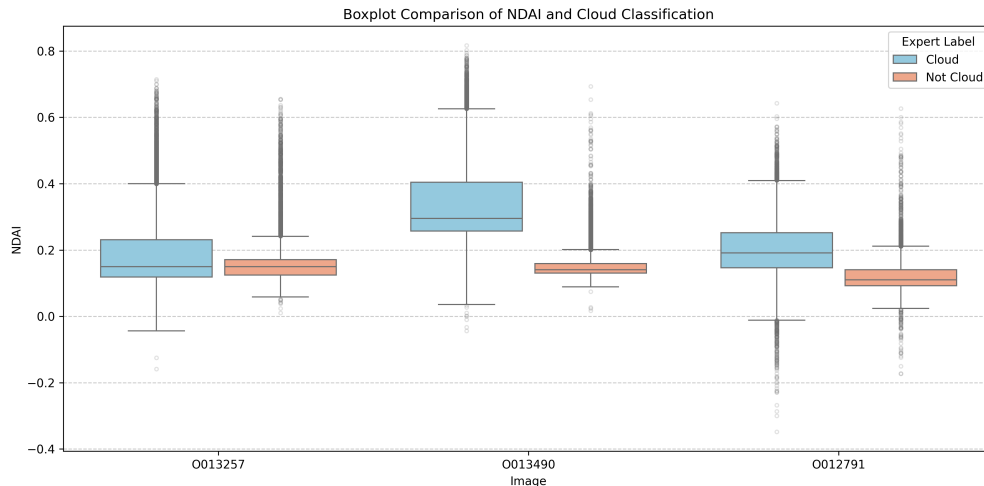


Figure 7: Distribution of NDAI for each labeled image

Figure 7 shows the distribution of NDAI across labeled images. In image 0013257, the median NDAI values for cloud and non-cloud pixels are about equal. In image 0013460, the NDAI values for cloud pixels are significantly higher than non-cloud pixels. Image 0012791 sees a similar pattern, but not as extreme. While NDAI values for cloud pixels varies by image, NDAI values for non-cloud pixels are centered around 0.1 with low variance. Consequently, values deviating strongly from this center are more likely to belong to clouds.

According to heatmap in Figure 2, CORR has essentially zero correlation with cloud label, which made us question why the domain experts would choose to create this feature. Through further data exploration, we found an interesting interaction between CORR and SD.

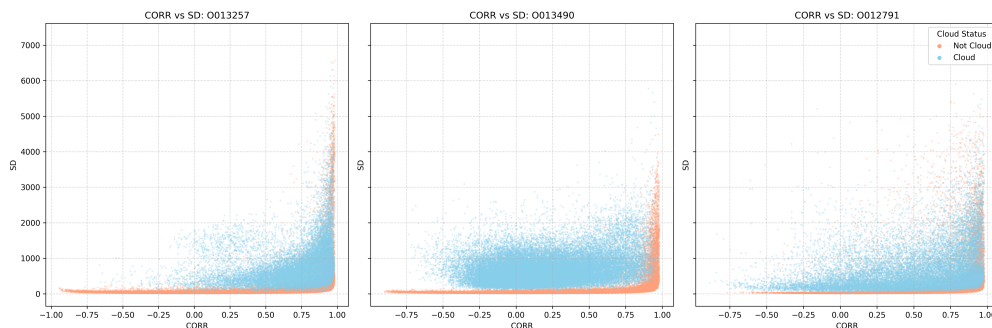


Figure 8: Scatterplot of CORR versus SD, colored by cloud label

The scatterplot in Figure 8 shows distinct regions of points based on their cloud classification. **Cloud pixels tend to have SD values very close to 0 or CORR values very close to 1.** This is shown by the

orange bands on the bottom and right sides of the scatterplot.

3.5 Splitting Data

We split the data at the image level for training and test sets, combined with 5-fold cross-validation over the training set. We held out image O013490 as the test set, and used images O013257 and O012791 for training.

This strategy is justified by several considerations:

- **Preventing data leakage.** Satellite images has strong spatial autocorrelation, meaning that nearby pixels often have similar features. If we split the data randomly at the pixel level, neighboring pixels could appear in both training and test sets, leading to data leakage and overestimation of model performance. To avoid this, we used an image level split.
- **Maximizing data utility.** As we only have three labeled images, we can not perform classic 60/20/20 split. Assigning one image each for training, validation, and testing would form an extremely small training set, which prevents the model from learning patterns that apply to unseen data. Instead, we apply 5-fold cross-validation on the training pool of two images to maximize the data for learning, while ensuring a robust estimate.
- **Reflecting real-world challenges.** In practice, the cloud detection model will face entirely new orbits, where new cloud patterns may arise. Holding out an image with the largest contiguous cloud systems as the test set can simulate the real-world challenges.

4 Feature Engineering

4.1 Identification of Informative Features

To determine which features are most useful for cloud classification, we evaluated all eight available features using three metrics: the area under the ROC curve (AUC) when using each feature alone as a classifier, mutual information with the binary cloud label, and the absolute Welch’s t -statistic measuring the difference in class means. These metrics were computed on all labeled pixels across all three images, after removing unlabeled pixels. Table 1 and Figure 9 summarize the results.

Table 1: Feature importance metrics. Features are ranked by average rank across all three metrics.

Feature	AUC	MI	$ t $	Avg Rank
SD	0.935	0.424	250.0	1.3
NDAI	0.823	0.252	263.6	1.7
angle_af	0.799	0.186	232.3	3.0
angle_an	0.794	0.177	226.9	4.0
angle_bf	0.770	0.161	202.7	5.0
angle_cf	0.643	0.082	113.0	6.3
angle_df	0.554	0.111	25.9	7.0
CORR	0.524	0.008	26.0	7.7

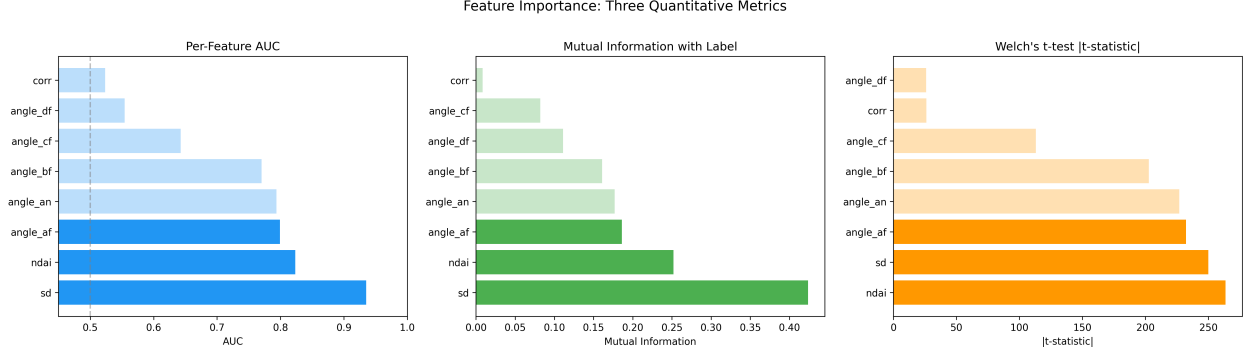


Figure 9: Feature importance across three metrics. Top three features highlighted in darker colors.

SD is the strongest single predictor with the highest AUC (0.935) and mutual information (0.424). **NDAI ranks second** (AUC = 0.823), consistent with the domain knowledge described by Shi et al. (2008). Among the raw radiance angles, **angle_af ranks third** (AUC = 0.799), although all five radiance features are highly correlated ($r > 0.81$), so the specific ranking among them is somewhat arbitrary.

CORR performs poorly on its own (AUC = 0.524, barely above chance). This is because CORR was designed to work together with SD and NDAI thresholds rather than as a standalone predictor, as shown in the CORR versus SD scatterplot in Figure 8.

The p -values from Welch’s t -test are effectively zero for all features, which is expected given the sample size (over 200,000 pixels) and spatial correlation among neighboring pixels. The $|t|$ -statistics are used here only for ranking, not for formal hypothesis testing.

Figure 10 visualizes the distribution of the top three features in different classes. SD has the least overlap between classes, this may explain why SD performs better than other features. The right tail of the angle_af distribution also indicates that cloud pixels dominate higher radiance values.

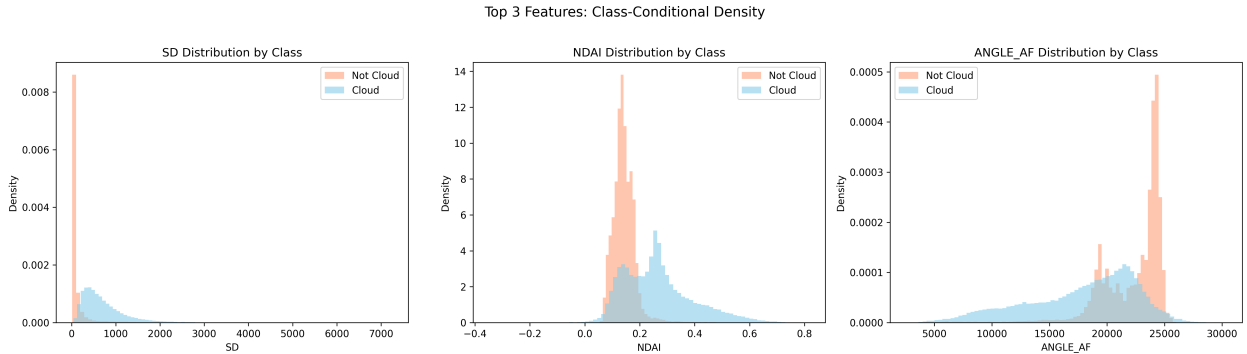


Figure 10: Class conditional density plots for the top three features.

To check whether these rankings are stable across images, we computed the AUC for each feature on each image separately. Figure 11 presents these results as a heatmap. SD is the most stable feature, with AUC above 0.88 on all three images (standard deviation = 0.033). NDAI shows high variability: it reaches 0.979 on O013490 but drops to 0.547 on O013257 (standard deviation = 0.179). This instability motivates the spatial features we engineer below.

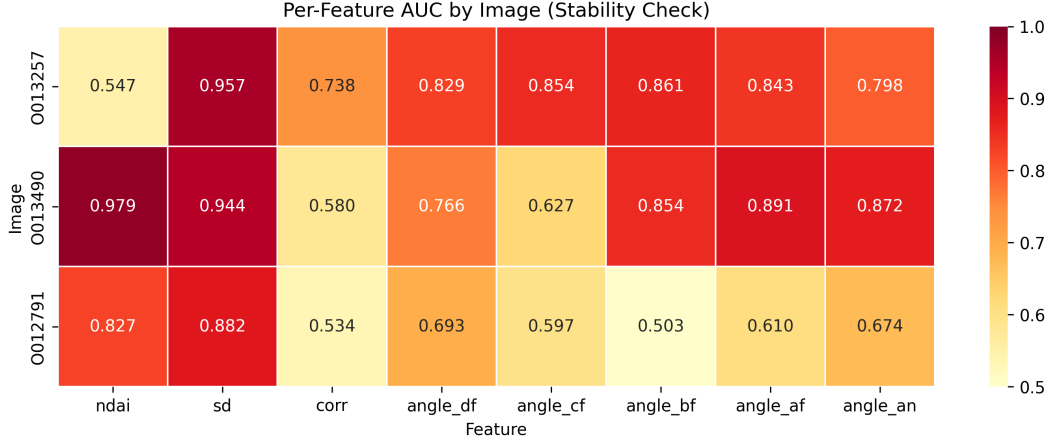


Figure 11: Per feature AUC by image. SD is stable across all images while NDAI varies substantially.

4.2 Engineered Features

The three expert features (NDAI, SD, CORR) may not fully exploit information from surrounding pixels. Combined with the instability of NDAI observed above, this motivated us to create eight new features in three categories.

Angular Features

These are computed per pixel from the five radiance measurements without spatial context:

- `rad_ratio`: ratio of Df to An radiance, capturing angular scattering similarly to NDAI but from raw measurements.
- `rad_range`: difference between maximum and minimum radiance across the five angles.
- `rad_std`: standard deviation of radiance across angles.

Spatial Features

These use a 5×5 pixel neighborhood, directly addressing the suggestion to use surrounding pixel information:

- `ndai_local_mean`: average NDAI in a 5×5 window (smoothed, denoised version of NDAI).
- `ndai_local_std`: standard deviation of NDAI in a 5×5 window, capturing local texture and heterogeneity near cloud boundaries.
- `sd_local_mean`: average SD in a 5×5 window.
- `sd_local_std`: standard deviation of SD in a 5×5 window.

Edge Feature

- `ndai_gradient`: Sobel gradient magnitude of NDAI, highlighting regions where NDAI changes rapidly such as cloud boundaries.

Evaluation

We used the same metrics to evaluate these engineered features. Table 2 and Figure 12 compare the predictive ability of the original and engineered features.

Table 2: Original vs engineered features. Engineered features marked with †. Stability is the standard deviation of per image AUC (lower is better).

Feature	AUC	Stability (std)	Best AUC	Worst AUC
SD	0.935	0.033	0.957	0.882
sd_local_mean†	0.936	0.031	0.958	0.885
ndai_local_std†	0.929	0.011	0.944	0.919
sd_local_std†	0.915	0.036	0.953	0.865
ndai_gradient†	0.893	0.011	0.905	0.878
NDAI	0.823	0.179	0.979	0.547
ndai_local_mean†	0.833	0.181	0.985	0.551
rad_ratio†	0.823	0.179	0.979	0.547
rad_range†	0.775	0.165	0.985	0.583
rad_std†	0.770	0.168	0.985	0.575

The spatial features are the clear winners. `sd_local_mean` (AUC = 0.936) slightly outperforms raw SD while maintaining similar stability. The most important new finding is `ndai_local_std` (AUC = 0.929), which achieves close to the predictive power of SD while being far more stable across images (standard deviation = 0.011 versus 0.179 for raw NDAI). This texture feature captures the variability at cloud boundaries, which is a robust signal regardless of the specific image. `ndai_gradient` (AUC = 0.893) shows similar stability (standard deviation = 0.011), confirming that edge information is a useful and consistent signal.

The angular features (`rad_ratio`, `rad_range`, `rad_std`) did not improve much over the original radiance features and showed the same instability across images. This is expected because they are simple transformations of the raw radiance without spatial context.

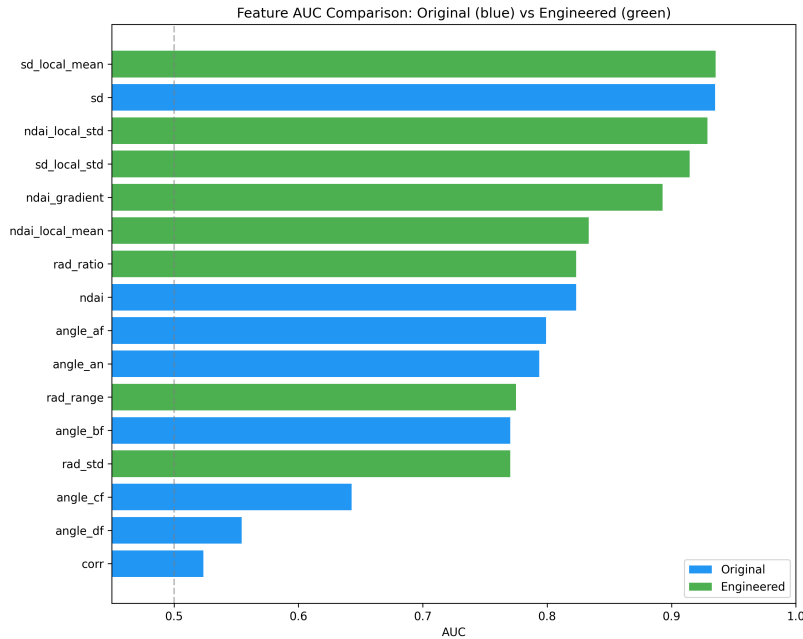


Figure 12: AUC comparison of original features (blue) and engineered features (green).

The class-conditional densities in Figure 13 reveal why spatial features perform well. `sd_local_mean` shows a clear separation between classes.

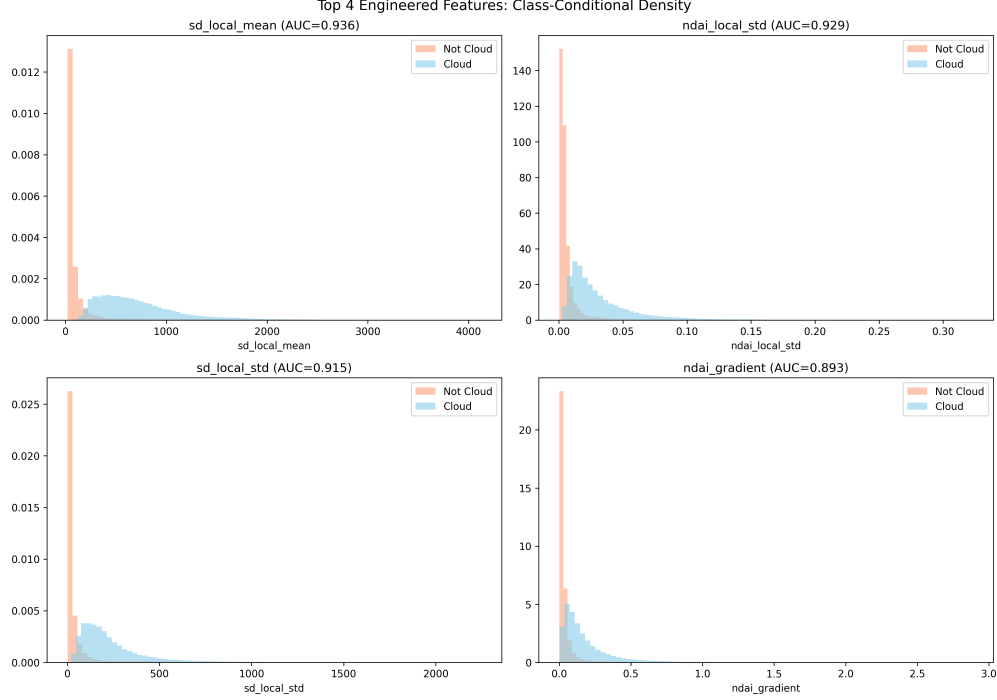


Figure 13: Class conditional density plots for the top four engineered features.

4.3 Transfer Learning via Autoencoder

We pre-trained an autoencoder on all 164 MISR images (including the 161 without expert labels) to learn compressed representations of 9×9 pixel patches. The autoencoder picks up spatial patterns from the larger pool of unlabeled data, and the resulting encoded features can then help the downstream classifiers.

Architecture

We replaced the provided MLP based autoencoder with a convolutional architecture that preserves the spatial structure of the input patches. The encoder uses three convolutional layers with batch normalization and LeakyReLU activations, progressively reducing the $8 \times 9 \times 9$ input to a $128 \times 3 \times 3$ feature map, then mapping it to a 16 dimensional embedding through a linear layer. The decoder mirrors this structure using transposed convolutions. The full model has 227,928 trainable parameters.

Training

We trained for 100 epochs using Adam (learning rate 5×10^{-4}) with a learning rate scheduler that halves the rate when validation loss plateaus. The loss function is mean squared error between the input patch and its reconstruction. To avoid data leakage, we split the data by image (131 train, 33 validation) rather than by pixel, ensuring the model is evaluated on entirely unseen satellite scenes.

All 164 images together contain approximately 19 million patches. Extracting every 9×9 patch up front would require roughly 47 GB of memory, which exceeds the available GPU memory. To solve this, we implemented a memory efficient data pipeline that stores only the full images and extracts patches on the fly during training, reducing memory usage to about 600 MB. Training was performed on a single NVIDIA V100 GPU (32 GB) on the PSC Bridges 2 cluster and completed in approximately 7 hours.

Embedding Extraction

After training, we passed each pixel in the three labeled images through the encoder to obtain 16 dimensional embeddings (ae0 through ae15). These capture spatial patterns learned from the full dataset and serve as additional input features for classification.

To visualize how well the autoencoder separates the two classes, we applied t-SNE to project the 16 dimensional embeddings down to two dimensions.

Figure 14 shows that cloud and non-cloud pixels form visually distinct clusters, even though the autoencoder was trained without any labels.

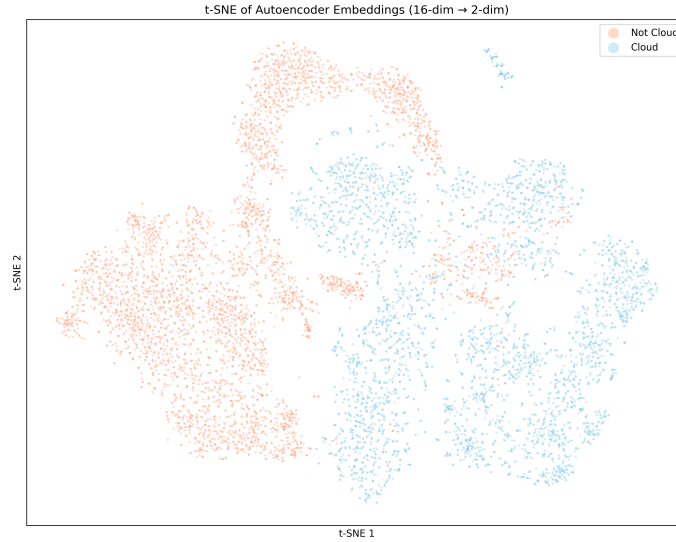


Figure 14: t-SNE of autoencoder embeddings colored by expert label. The separation between cloud and non-cloud clusters shows that the unsupervised autoencoder learned representations useful for classification.

Figure 15 splits the visualization by images. All of them show separation between cloud and non-cloud class, confirming that the learned features are stable and useful.

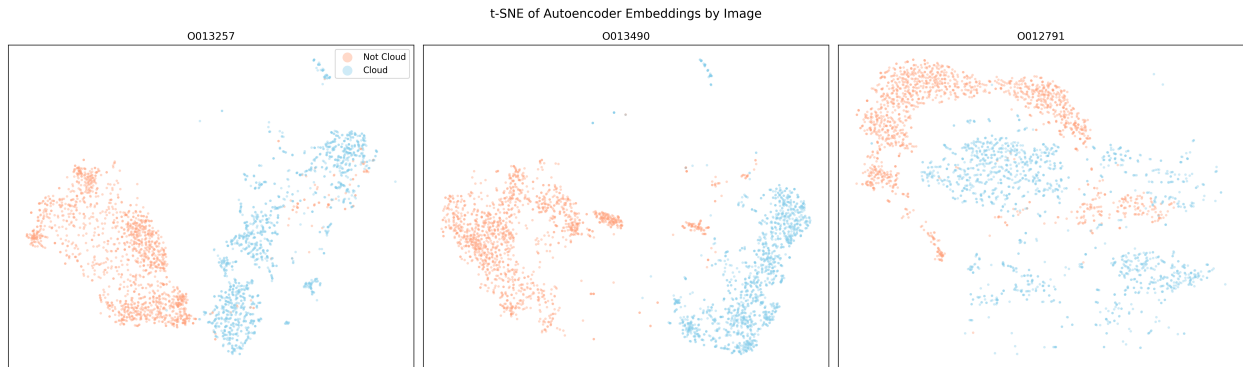


Figure 15: t-SNE of autoencoder embeddings split by image. Cloud versus non-cloud separation is consistent across all three images.

Figure 16 shows that the embeddings from different images overlap substantially, indicating that the autoencoder learned generalizable features rather than image specific ones. This is evidence that the transfer learning from unlabeled images was effective.

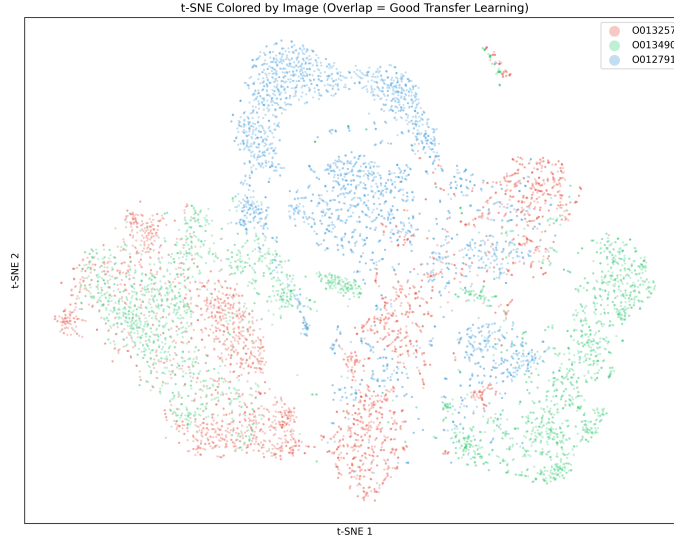


Figure 16: t-SNE colored by source image. The overlap among images indicates the autoencoder learned shared features across different satellite scenes rather than image specific patterns.

5 Modeling

5.1 Classifier Descriptions and Assumptions

We constructed a feature matrix of 32 dimensions, using 8 original features, 8 engineered features and 16 autoencoder features.

Three classifiers were implemented.

Logistic Regression

We used logistic regression as our baseline model because of its simplicity and ability to be trained quickly. It is also highly interpretable, since it assigns a coefficient to each feature that relates to its importance. The model assumes the log-odds of the label is a linear combination of the features. Actually, based on Figure 8, the true relationship is nonlinear, so the strict linearity assumption is violated. We used SAGA solver to accelerate the algorithm. Since the logistic regression did not converge with the default settings, we set the maximum iterations to 10,000 and the tolerance to 0.005.

Random Forest

Random Forests aggregate the predictions of an ensemble of decision trees. As a non-parametric model, it makes no assumptions about the distributions of variables or linear relationships. We experimented with different random forests of varying complexities. More specifically, the parameters and values we explored were `{n_estimators: [100, 300, 500], max_depth: [20, 40, None], min_samples_leaf: [1, 5, 20]}`.

- `n_estimators`: The number of individual decision trees in the ensemble.
- `max_depth`: The maximum depth of each tree. Higher value results in a tree with more splits and more complex decision boundaries.
- `min_samples_leaf`: The minimum samples required to form a new leaf node. Larger values will regularize each tree, which reduces overall complexity.

To find the best combination of parameters for the random forest, we performed 5-fold cross validation grid search using ROC-AUC as our metric. In our parameter search, we allowed individual trees to potentially be very complex (high values of `max_depth` and small values of `min_samples_leaf`). We felt that this would be counteracted by the fact that the decisions are aggregated across many trees.

The best-performing Random Forest model has a mean CV AUC of 0.9897 with a standard deviation of

0.0103 across folds. This model has parameters: max_depth: 40, min_samples_leaf: 1, n_estimators: 500.

Gradient Boosting

This is another tree-based ensemble method, with the difference being that it fits trees sequentially to correct previous errors. It is also a non-parametric model and makes no assumptions about the variables. Similarly, we ran a 5-fold cross validation grid search to identify the best combination of parameters. The parameter space we explored was {max_iter: [100, 300, 500], max_depth: [5, 10, None], learning_rate: [0.01, 0.05, 0.1], min_samples_leaf: [1, 5, 20]}

- max_iter: The number of boosting rounds (similar to n_estimators in random forest).
- max_depth: The maximum depth of each tree.
- learning_rate: Determines how much each new tree contributes to the final ensemble. Higher learning rate requires fewer trees to converge. Smaller learning rate requires fewer trees but allows for more precise training.
- min_samples_leaf: The minimum samples required to form a new leaf node.

Since trees are added sequentially to correct previous errors, the model is more prone to overfitting. Therefore, we looked at parameters that resulted in weaker trees compared to random forest. One benefit of Gradient Boosting is that it takes advantage of early stopping, so it can train much faster than Random Forest.

The best-performing Gradient Boosting model has a mean CV AUC of 0.9911 with a standard deviation of 0.0108 across folds, which is nearly identical to Random Forest. This model has parameters: {max_iter: 500, max_depth: 10, learning_rate: 0.1, min_samples_leaf: 50}.

5.2 Model Evaluation

We evaluated the models using ROC and AUC instead of accuracy, because image blocks are often imbalanced (e.g. dominated by clouds), and a model with high accuracy could still fail to classify minor classes correctly. For Random Forest and Gradient Boosting, we evaluated based on their best-performing parameters.

The way in which we split the data is described in the EDA section. Within the training pool, we used a 5-fold block-based cross-validation. Each training image was divided into 9×9 blocks to prevent spatial data leakage.

Table 3 summarizes the cross-validation results and the performance of the test set, and Figure 17 shows ROC curves on the test set.

Table 3: Cross-validation results and test set performance.

Classifier	CV AUC $\pm$ std	Test AUC
Logistic Regression	0.9205 ± 0.0295	0.9701
Random Forest	0.9897 ± 0.0103	0.9929
Gradient Boosting	0.9911 ± 0.0108	0.9978

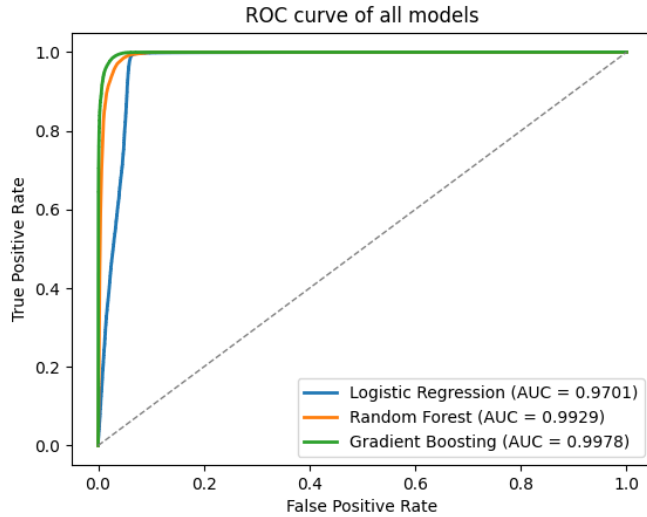


Figure 17: ROC curves on the test set.

Based on CV AUC, Gradient Boosting gives the best results, while Random Forest performs similarly. Logistic Regression performs worse than the tree-based models, suggesting that the relationship between the features and the cloud labels is likely nonlinear. Since the performance of Gradient Boosting and Random Forest is comparable, we chose Gradient Boosting as our preferred classifier since it has lower complexity. This is supported by the fact that Gradient Boosting has a `max_depth` of 10 and `min_samples_leaf` of 50 compared to Random Forest with a `max_depth` of 40 and `min_samples_leaf` of 1. Therefore, we were worried that Random Forest could overfit to the training data while Gradient Boosting could generalize better to unseen data.

As shown by the test AUC scores in Table 3 and the ROC curves in Figure 17, our preferred Gradient Boosting classifier indeed performs the best on unseen data.

5.3 Diagnostics of the Best Model

To examine where our best model makes errors, we make predictions on the test set and generate a confusion matrix, using a 0.5 threshold. Here, we can see that there are about as many cloud pixels as non-cloud pixels. Moreover, we have relatively few misclassifications (as the high AUC mentioned above suggests) with the majority of the errors being misclassifications of not clouds as clouds. The confusion matrix, along with the diagnostics discussed below, is displayed in Figure 19.

We also construct a precision-recall curve to study the tradeoff between false positives and false negatives. Average precision is 0.9978, meaning the model can simultaneously achieve near-perfect precision and recall across all thresholds. Precision stays above 0.99 until recall exceeds ~ 0.99 , at which point it drops sharply. Hence, there is a small set of hard cloud pixels the model cannot recover without admitting many false positives. To study these pixels in more detail, we construct an error map that shows which pixels exactly were predicted correctly or incorrectly and with which confidence. This map is displayed in Figure ???. We can see that the majority of pixels have predicted probabilities of 0 or 1. Errors seem to cluster along cloud boundaries. Interior cloud and non-cloud regions are classified almost perfectly and the errors occur at the transitions between those two areas.

Next, we construct a calibration curve, i.e. we examine how well calibrated the probabilities are that the model returns. In a well calibrated model, if we look at the fraction of clouds among all pixels that were assigned a 10% probability for the pixel being a cloud, we would find that approximately 10% are indeed clouds. Our model, on the other hand is not well calibrated. In particular, it is over-confident especially at intermediate predicted probabilities. A predicted probability of 80% corresponds to a less than 40% true-positive rate. This means the raw predicted probabilities should not be taken at face value as probability

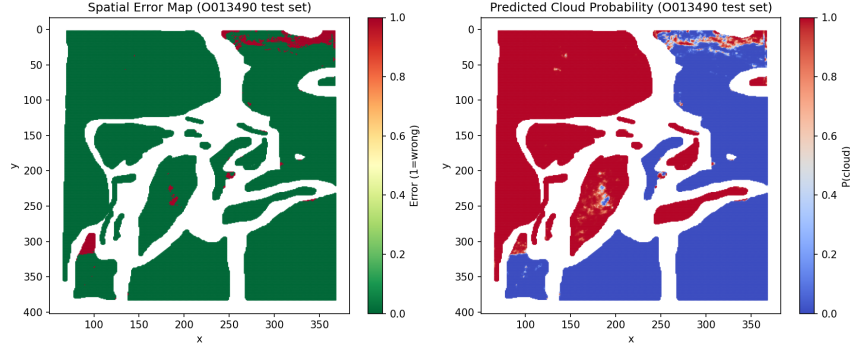


Figure 18: Spatial Error Map on Test Set

estimates, even though, as the AUC suggests, model’s ranking of pixels by confidence is reliable.

As part of diagnostics we also consider feature importance. Specifically, for each feature, one-by-one, we randomly permute that feature’s values across all test pixels and measure how much the test AUC drops relative to the unmodified baseline. This is repeated 10 times per feature. We then report the average drop. Importantly, the model is never retrained, only the values in the feature column that we want to study change. If a feature is genuinely useful, permuting randomly the values should degrade predictions. Here, we find that `sd_local_mean` is by far the most important feature. Without this feature, our AUC drops by about 0.13. Several of the autoencoder embeddings (`ae13`, `ae4`, `ae11`, `ae3`, `ae15`) come next, followed by `corr`, `angle_cf` and `sd_local_std`. NDAI does not appear in the top 15, consistent with its instability across images noted in Section 4.

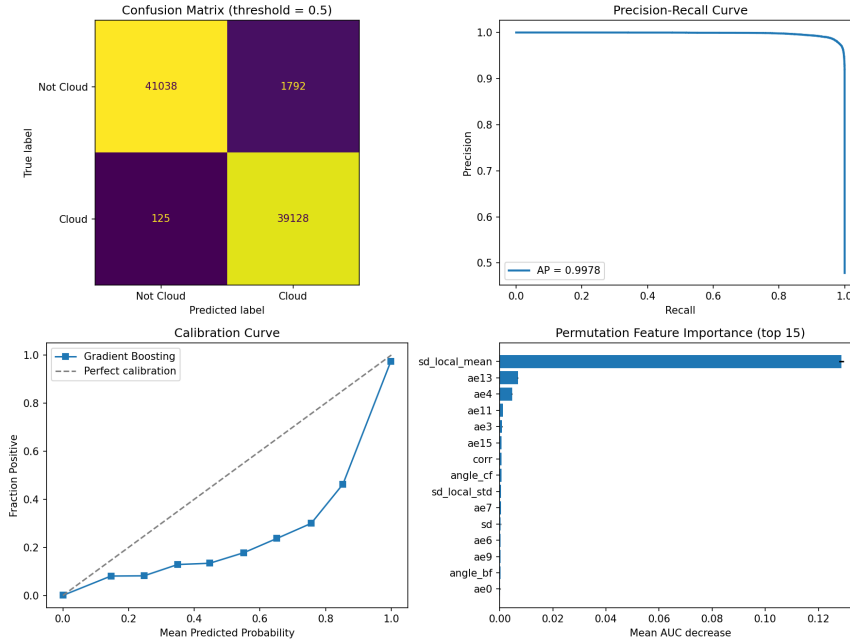


Figure 19: Diagnostic Summary

We also study how fast AUC converges on the train and test set to determine how many boosting iterations are needed until further training benefit goes to zero. Figure 20 shows that training AUC converges quickly, while test AUC fluctuates with small variance around 0.998 after 200 iterations. The performance of test set does not get worse when iterations increase, so there is currently no indication of overfitting. However, the

plot suggests that we could reduce compute by using only about 200 boosting iterations while maintaining similar test set AUC.

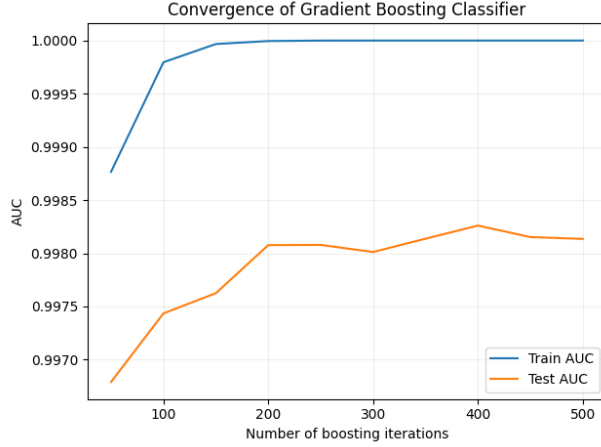


Figure 20: Relationship between AUC and the number of iterations.

5.4 Stability Analysis

In line with the PCS framework, we perform stability analyses to examine how stable our conclusions are to different forms of perturbations. Firstly, we study whether the choice of images that we use for training and testing affects the ranking of our model, i.e. whether we would have chosen a different "best model" if we had used a different train-test split. Here, we find that our ranking of preferred models is stable to the choice of specific train and test sets and hence the model selection conclusion is stable with respect to this perturbation. The results of this analysis are shown in Table 4.

Table 4: Test AUC for each model across all three leave-one-out image splits.

Test image	Logistic Regression	Random Forest	Gradient Boosting
O013490	0.9701	0.9928	0.9981
O013257	0.7101	0.9842	0.9862
O012791	0.7829	0.7725	0.9137

One thing to notice is that the Test AUC varies widely depending on the images we use for training and testing. In particular, it seems as if clouds would be harder to predict accurately on 0012791. Here, gradient boosting AUC drops to around 0.91. Thus, the strong performance of our model is not guaranteed to generalize to all satellite images.

Second, similarly, we examine stability of the model selection conclusion with respect to the feature set. To do this, we consider three monotonically increasing sets of features, i.e. we consider three feature sets that we can order by inclusion. The feature sets are the original 8 features, the original 8 together with the 8 engineered features, and, lastly, those 16 features plus the 16 embeddings. Here, again, we find that irrespective of which of those three feature sets we use, the gradient boosting approach performs the best in terms of Test AUC. The results of this analysis are displayed in Table 5.

Last, we consider sensitivity to training data volume. Other researchers interested in predicting cloud/non-cloud pixels might have different data sizes to train on. For those, it would be interesting whether the conclusion that gradient boosting is the best performing model we consider is robust to the size of the training data. Hence, we re-train our model on different fractions of the original training data and study how Test AUC behaves as a function of training data size. Here, we find that even with 10% of the original data (approx. 12k pixels), we reach the same test set AUC as in our full sample. Hence, even with a small

Table 5: Test AUC on 0013490 for all three models across three nested feature sets.

Feature set	Logistic Regression	Random Forest	Gradient Boosting
Original (8)	0.9707	0.9875	0.9950
+ Engineered (16 total)	0.9701	0.9857	0.9948
+ AE embeddings (32 total)	0.9701	0.9928	0.9981

fraction of our training data gradient boosting performs equally well. The results of this are displayed in Table 6.

Table 6: Gradient Boosting test AUC on 0013490 as a function of training set size.

Training fraction	n pixels	AUC
10%	12,559	0.9986
25%	31,399	0.9974
50%	62,799	0.9974
75%	94,198	0.9980
100%	125,598	0.9979

6 Conclusion

In this report, we train machine learning models for predicting whether pixels on satellite imagery are clouds or not clouds. To this end, we work with MISR data from three expert-labeled satellite images and a pool of 161 unlabeled images.

Our analysis proceeded in several stages. We began our analyses with extensive EDA and identified SD and NDAI most discriminative features, though with the caveat that NDAI is unstable across images. To address this instability and enrich the feature space, we engineered eight spatial and angular features. The most valuable of those features were `sd_local_mean` and `ndai_local_std`. We also trained a convolutional autoencoder on all 164 images to extract 16 transfer-learned embeddings per pixel. Together these give a 32-dimensional feature matrix that we use for our predictive modeling.

In our modeling, we compared three types of classifiers: logistic regression as a baseline, random forest, and gradient boosting. Both tree-based models substantially outperform logistic regression, indicating that the relationship between features and cloud labels is nonlinear that makes tree-based models more suitable for our prediction task. Gradient boosting achieves the highest test AUC of 0.9981 on the held-out image (0013490), performing slightly better than the random forest model (0.9928) while using shallower trees (`max_depth` = 10 vs. 40), which gives us more confidence that it will generalize well. Overall, we proposed that the gradient boosting model is the most appropriate model among the models that we considered.

Diagnostics reveal that gradient boosting model’s errors are heavily concentrated at cloud boundaries. Pixels in the interior or cloud and non-cloud areas are predicted overwhelmingly correctly and with high confidence. The main error our model is making are false positives, i.e. predicting non-cloud pixels as clouds. The boundary pixels that the model is falsely classifying as clouds have anomalously high local radiance variance values (`sd_local_mean` $\approx$ 983 on average). We suggest that this might be one of the reasons that the model mistakes them for clouds.

Our stability analyses show that the model selection conclusion, i.e. that gradient boosting is best performing model among the ones considered, holds across all three possible leave-one-out image splits and across different feature sets. However, the absolute performance is less stable: when testing on 0012791, gradient boosting AUC drops to 0.914 and random forest to 0.773, suggesting that some satellite images contain cloud patterns that are harder to generalize to. The learning curve analysis shows that performance saturates at around 10% of the training data, implying that further labeled images would not substantially improve results on 0013490 specifically and that the results are stable with respect to the size of data; 10% of the data is enough to achieve the same performance.

A key limitation of this work is the small number of labeled images (three in total). The variability in performance across images means that our reported test AUC of 0.998 may overstate how well the model would perform on a broader set of orbits and is cause for worry. In future work, a larger number of test images that reflects the composition that would be observed in the actual application would be desirable. Another limitation is that the probability estimates that the model produces are not well calibrated, as we show in the diagnostics section. Hence, in future iterations of the model, we may want to address this miscalibration to yield results that are interpretable as actual cloud probabilities.

References

- Shi, T., Yu, B., Clothiaux, E. E. & Braverman, A. J. (2008), ‘Daytime arctic cloud detection based on multi-angle satellite data with case studies’, *Journal of the American Statistical Association* **103**(482), 584–593.

A Academic honesty

A.1 Statement

We affirm that the work was done by ourselves. The external sources are properly cited.

Academic research honesty is necessary because dishonest behavior can undermine the reliability of research. We need to respect the contributions of others and maintain fairness in the field of research.

A.2 LLM Usage

Coding

We used GitHub Copilot with autocomplete suggestions to improve the code and figures. The main code is generated by ourselves.

Writing

We used LLMs to check grammar only.